Anexos Técnicos - Arquitectura PeluDog CRM

Este documento contiene todas las configuraciones técnicas, código y ejemplos de implementación para la arquitectura establecida del sistema PeluDog CRM.

A1. Configuración de ActiveStorage

A1.1 Configuración de Storage Local

```
# config/storage.yml
local:
  service: Disk
  root: <%= Rails.root.join("storage") %>

# config/environments/production.rb
config.active_storage.variant_processor = :image_processing
config.active_storage.resolve_model_to_route = :rails_storage_proxy
```

A1.2 Configuración para S3 (Escalamiento)

```
# config/storage.yml
amazon:
  service: S3
  access_key_id: <%= ENV['AWS_ACCESS_KEY_ID'] %>
  secret_access_key: <%= ENV['AWS_SECRET_ACCESS_KEY'] %>
  region: <%= ENV['AWS_REGION'] %>
  bucket: <%= ENV['AWS_BUCKET'] %>

# config/environments/production.rb
config.active_storage.service = :amazon
```

A1.3 Migración de Storage

```
# lib/tasks/storage_migration.rake
namespace :active_storage do
  desc "Migrate all files from local to S3"
  task migrate_to_s3: :environment do
    ActiveStorage::Blob.find_each do |blob|
      if blob.service_name != 's3'
        blob.open do |file|
          blob.upload_without_unfurling(file)
          blob.update!(service_name: 's3')
        end
      end
```

```
    end
  end
end
```

A2. Sistema de Backups con Whenever

A2.1 Configuración del Gem

```
# Gemfile
gem 'whenever', require: false

# config/schedule.rb
set :output, "log/cron.log"
set :environment, Rails.env

every 1.day, at: '2:00 am' do
  rake "db:backup"
end

every 1.day, at: '2:30 am' do
  rake "db:cleanup_backups"
end
```

A2.2 Rake Tasks de Backup

```
# lib/tasks/backup.rake
namespace :db do
  desc "Create database backup"
  task backup: :environment do
    config = Rails.application.config.database_configuration[Rails.env]
    timestamp = Time.current.strftime("%Y%m%d_%H%M%S")
    backup_dir = Rails.root.join('db', 'backups')
    FileUtils.mkdir_p(backup_dir)

    backup_file = "#{backup_dir}/peludog_backup_#{timestamp}.sql"

    cmd = "mysqldump -h #{config['host']} -u #{config['username']} " \
          "-p#{config['password']} #{config['database']} > #{backup_file}"

    system(cmd)
    system("gzip #{backup_file}")

    puts "Backup creado: #{backup_file}.gz"
  end

  desc "Restore database from backup"
  task :restore, [:backup_file] => :environment do |t, args|
    config = Rails.application.config.database_configuration[Rails.env]
```

```
    backup_file = args[:backup_file]

    raise "Backup file not found" unless File.exist?(backup_file)

    if backup_file.end_with?('.gz')
      system("gunzip -c #{backup_file} > tmp_restore.sql")
      backup_file = 'tmp_restore.sql'
    end

    cmd = "mysql -h #{config['host']} -u #{config['username']} " \
          "-p#{config['password']} #{config['database']} < #{backup_file}"

    system(cmd)
    File.delete('tmp_restore.sql') if File.exist?('tmp_restore.sql')

    puts "Base de datos restaurada desde: #{args[:backup_file]}"
  end

  desc "Clean old backups (keep last 6)"
  task cleanup_backups: :environment do
    backup_dir = Rails.root.join('db', 'backups')
    backups = Dir.glob("#{backup_dir}/*.sql.gz").sort

    if backups.size > 6
      old_backups = backups[0...-6]
      old_backups.each do |backup|
        File.delete(backup)
        puts "Backup eliminado: #{backup}"
      end
    end
  end
end
```

A3. Configuraciones Docker

A3.1 Docker Compose Principal

```
version: '3.8'

services:
  nginx:
    image: nginx:alpine
    ports:
      - "80:80"
      - "443:443"
    volumes:
      - ./nginx.conf:/etc/nginx/nginx.conf
      - ./ssl:/etc/ssl/certs
      - static_files:/app/public
    depends_on:
```

```
      - backend
      - frontend
    networks:
      - peludog_network

  backend:
    build:
      context: ./backend
      dockerfile: Dockerfile
    env_file:
      - .env.production
    volumes:
      - storage_data:/app/storage
      - backup_data:/app/db/backups
    depends_on:
      - mysql
    networks:
      - peludog_network

  frontend:
    build:
      context: ./frontend
      dockerfile: Dockerfile
    volumes:
      - static_files:/app/dist
    networks:
      - peludog_network

  mysql:
    image: mysql:8.0
    env_file:
      - .env.production
    volumes:
      - mysql_data:/var/lib/mysql
      - ./mysql/conf.d:/etc/mysql/conf.d
    ports:
      - "3306:3306"
    networks:
      - peludog_network

volumes:
  mysql_data:
  storage_data:
  backup_data:
  static_files:

networks:
  peludog_network:
    driver: bridge
```

A3.2 Docker Compose para Escalamiento

```yaml
# docker-compose.scale.yml
version: '3.8'

services:
  nginx-lb:
    image: nginx:alpine
    ports:
      - "80:80"
      - "443:443"
    volumes:
      - ./nginx-lb.conf:/etc/nginx/nginx.conf
      - ./ssl:/etc/ssl/certs
    depends_on:
      - backend
    networks:
      - peludog_network

  backend:
    build:
      context: ./backend
      dockerfile: Dockerfile
    env_file:
      - .env.scaling
    volumes:
      - shared_storage:/app/storage
      - backup_data:/app/db/backups
    depends_on:
      - mysql
      - redis
      - storage-server
    networks:
      - peludog_network

  storage-server:
    image: minio/minio
    env_file:
      - .env.scaling
    volumes:
      - minio_data:/data
    command: server /data --console-address ":9001"
    ports:
      - "9000:9000"
      - "9001:9001"
    networks:
      - peludog_network

  redis:
    image: redis:7-alpine
    volumes:
      - redis_data:/data
    networks:
      - peludog_network
```

```
volumes:
  shared_storage:
  minio_data:
  redis_data:
```

A4. Configuraciones de Aplicación Rails

A4.1 Configuración de Puma (Escalamiento Vertical)

La concurrencia en Puma se gestiona mediante una combinación de **Workers** (procesos) y **Threads** (hilos). La capacidad total de peticiones simultáneas se calcula así: `(Nº de Workers) * (Nº de Threads)`.

- **Workers (`WEB_CONCURRENCY`):** Son clones del proceso de la aplicación. Cada uno consume su propia memoria y puede ejecutarse en un core de CPU distinto. Es la forma principal de escalar para aprovechar servidores multi-core.
- **Threads (`RAILS_MAX_THREADS`):** Permiten que cada Worker maneje múltiples peticiones a la vez. Mientras un hilo espera por la base de datos, otro puede procesar una nueva petición.

La configuración se realiza a través de variables de entorno para no tener que modificar el código:

> **Nota de Configuración:** Los valores por defecto en este ejemplo (`WEB_CONCURRENCY=1`, `RAILS_MAX_THREADS=5`) son un punto de partida seguro para el hardware **Básico (Inicial)**, resultando en 5 peticiones simultáneas. A medida que el servidor escale, estos valores deben incrementarse. Un servidor con 4 cores y 8GB de RAM podría usar `WEB_CONCURRENCY=2` y `RAILS_MAX_THREADS=5` para una capacidad de 10 peticiones simultáneas.

```ruby
# config/puma.rb
workers_count = ENV.fetch('WEB_CONCURRENCY', 1).to_i
threads_count = ENV.fetch('RAILS_MAX_THREADS', 5).to_i

workers workers_count
threads threads_count, threads_count

port ENV.fetch('PORT', 3000)
environment ENV.fetch('RAILS_ENV', 'development')

worker_timeout 30
worker_boot_timeout 30

on_worker_boot do
  ActiveRecord::Base.establish_connection if defined?(ActiveRecord)
end

preload_app!
```

A4.2 Configuración de Base de Datos (Multi-servidor)

```
# config/database.yml
production:
  primary:
    adapter: mysql2
    encoding: utf8mb4
    pool: <%= ENV.fetch("RAILS_MAX_THREADS", 25) %>
    username: <%= ENV['DB_USERNAME'] %>
    password: <%= ENV['DB_PASSWORD'] %>
    host: <%= ENV['DB_PRIMARY_HOST'] %>
    database: <%= ENV['DB_NAME'] %>
    timeout: 5000

  replica:
    adapter: mysql2
    encoding: utf8mb4
    pool: <%= ENV.fetch("RAILS_MAX_THREADS", 25) %>
    username: <%= ENV['DB_USERNAME'] %>
    password: <%= ENV['DB_PASSWORD'] %>
    host: <%= ENV['DB_REPLICA_HOST'] %>
    database: <%= ENV['DB_NAME'] %>
    replica: true
    timeout: 5000
```

A4.3 Configuración de Redis y Cache (Para Escalamiento)

Nota: Esta configuración se aplica durante la fase de escalamiento, no en la instalación inicial.

```
# config/environments/production.rb
config.cache_store = :redis_cache_store, {
  url: ENV['REDIS_URL'],
  pool_size: ENV.fetch('RAILS_MAX_THREADS', 25),
  pool_timeout: 5
}

config.session_store :redis_store, {
  servers: [ENV['REDIS_URL']],
  expire_after: 1.week,
  key_prefix: 'peludog:session:'
}
```

A5. Configuraciones de NGINX

A5.1 NGINX Simple (Fase Inicial)

```
# nginx.conf
events {
    worker_connections 1024;
```

```
}

http {
    upstream backend {
        server backend:3000;
    }

    server {
        listen 80;
        server_name peludog.local;

        location / {
            root /app/public;
            try_files $uri @backend;
        }

        location @backend {
            proxy_pass http://backend;
            proxy_set_header Host $host;
            proxy_set_header X-Real-IP $remote_addr;
            proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
            proxy_set_header X-Forwarded-Proto $scheme;
        }
    }
}
```

A5.2 NGINX Load Balancer (Escalamiento)

```
# nginx-lb.conf
events {
    worker_connections 2048;
}

http {
    upstream backend {
        least_conn;
        server backend_1:3000 max_fails=3 fail_timeout=30s;
        server backend_2:3000 max_fails=3 fail_timeout=30s;
        server backend_3:3000 max_fails=3 fail_timeout=30s;
        keepalive 32;
    }

    upstream storage {
        server storage-server:9000;
    }

    server {
        listen 80;
        server_name peludog.com;

        client_max_body_size 50M;
```

```
        location / {
            root /app/public;
            try_files $uri @backend;
        }



        location @backend {
            proxy_pass http://backend;
            proxy_set_header Host $host;
            proxy_set_header X-Real-IP $remote_addr;
            proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
            proxy_set_header X-Forwarded-Proto $scheme;

            # Mantener conexiones activas
            proxy_http_version 1.1;
            proxy_set_header Connection "";
        }
    }
}
```

A6. Configuración MySQL para Escalamiento

A6.1 MySQL Master

```
# mysql/master.cnf
[mysqld]
server-id = 1
log-bin = mysql-bin
binlog-format = ROW
sync_binlog = 1
innodb_flush_log_at_trx_commit = 1

# Performance tuning
innodb_buffer_pool_size = 8G
innodb_log_file_size = 2G
max_connections = 400
query_cache_size = 512M
tmp_table_size = 1G
max_heap_table_size = 1G
```

A6.2 MySQL Replica

```
# mysql/replica.cnf
[mysqld]
server-id = 2
relay-log = relay-bin
```

```
read_only = 1
super_read_only = 1

# Performance tuning
innodb_buffer_pool_size = 6G
innodb_log_file_size = 1G
max_connections = 200
query_cache_size = 256M
```

A7. Comandos de Administración

A7.1 Gestión de Backups

```
# Crear backup manual
docker-compose exec backend rake db:backup

# Restaurar desde backup específico
docker-compose exec backend rake
db:restore[/app/db/backups/backup_20240815.sql.gz]

# Listar backups disponibles
docker-compose exec backend ls -la db/backups/

# Verificar estado de cron jobs
docker-compose exec backend crontab -l

# Instalar cron jobs
docker-compose exec backend whenever --update-crontab
```

A7.2 Escalamiento de Servicios

```
# Escalar backend a 3 instancias
docker-compose up --scale backend=3

# Ver estado de servicios
docker-compose ps

# Ver logs de una instancia específica
docker-compose logs backend_2

# Reiniciar solo el backend
docker-compose restart backend
```

A7.3 Gestión de Storage

```
# Limpiar archivos temporales
docker-compose exec backend rake active_storage:cleanup

# Ver uso de almacenamiento
docker-compose exec backend du -sh storage/

# Migrar archivos a S3
docker-compose exec backend rake active_storage:migrate_to_s3

# Verificar integridad de archivos
docker-compose exec backend rake active_storage:verify
```

A7.4 Monitoreo y Debugging

```
# Ver conexiones activas de MySQL
docker-compose exec mysql mysql -u root -p -e "SHOW PROCESSLIST;"

# Ver estado de Redis
docker-compose exec redis redis-cli info

# Ver métricas de Puma
docker-compose exec backend pumactl stats

# Ver logs en tiempo real
docker-compose logs -f backend
```

A8. Monitoring y Observabilidad

> **Nota sobre Monitoreo:** La siguiente configuración de Prometheus es una sugerencia para un monitoreo avanzado en etapas de escalamiento. Para la versión inicial del producto, el sistema de logs integrado de Rails (`log/production.log`) es más que suficiente para las tareas de monitoreo y debugging.

A8.1 Configuración de Prometheus

```
# prometheus/prometheus.yml
global:
  scrape_interval: 15s

scrape_configs:
  - job_name: 'peludog-backend'
    static_configs:
      - targets: ['backend:3000']
    metrics_path: '/metrics'

  - job_name: 'mysql'
```

```
    static_configs:
      - targets: ['mysql:3306']
```

A8.2 Métricas Custom en Rails

```
# config/initializers/prometheus.rb
require 'prometheus/client'

prometheus = Prometheus::Client.registry

# Métricas custom
$request_duration = prometheus.histogram(
  :http_request_duration_seconds,
  'HTTP request duration'
)

$database_query_duration = prometheus.histogram(
  :database_query_duration_seconds,
  'Database query duration'
)

$active_storage_uploads = prometheus.counter(
  :active_storage_uploads_total,
  'Total number of file uploads'
)
```

A9. Scripts de Deployment

A9.1 Script de Instalación Inicial

```
#!/bin/bash
# install.sh

echo "Instalando PeluDog CRM..."

# Crear directorios necesarios
mkdir -p ssl mysql/conf.d

# Generar certificados SSL auto-firmados para desarrollo
openssl req -x509 -newkey rsa:4096 -keyout ssl/key.pem -out ssl/cert.pem -
days 365 -nodes

# Copiar configuraciones
cp config/mysql/peludog.cnf mysql/conf.d/

# Construir e iniciar servicios
docker-compose build
docker-compose up -d
```

```
# Esperar a que MySQL esté listo
sleep 30

# Ejecutar migraciones
docker-compose exec backend rails db:create db:migrate db:seed

# Configurar cron jobs
docker-compose exec backend whenever --update-crontab

echo "¡PeluDog CRM instalado correctamente!"
echo "Accede a http://localhost para comenzar"
```

A9.2 Script de Escalamiento

```
#!/bin/bash
# scale.sh

BACKEND_INSTANCES=${1:-3}

echo "Escalando PeluDog a $BACKEND_INSTANCES instancias..."

# Crear configuración de load balancer
envsubst < nginx-lb.template > nginx-lb.conf

# Escalar servicios
docker-compose -f docker-compose.scale.yml up --scale
backend=$BACKEND_INSTANCES -d

echo "Sistema escalado correctamente"
docker-compose ps
```

A10. Testing y Validación

A10.1 Tests de Escalamiento

```
# test/integration/scalability_test.rb
require 'test_helper'

class ScalabilityTest < ActionDispatch::IntegrationTest
  test "sistema maneja 100 usuarios concurrentes" do
    threads = []

    100.times do |i|
      threads << Thread.new do
        post '/api/auth/login', params: {
          email: "user#{i}@test.com",
          password: 'password'
```

```
        }

        assert_response :success
      end
    end

    threads.each(&:join)
  end
end
```

A10.2 Tests de Backup y Restore

```
# test/tasks/backup_test.rb
require 'test_helper'

class BackupTaskTest < ActiveSupport::TestCase
  test "backup y restore funcionan correctamente" do
    # Crear datos de prueba
    user = User.create!(name: 'Test', email: 'test@test.com')

    # Ejecutar backup
    system('rake db:backup RAILS_ENV=test')

    # Limpiar base de datos
    User.destroy_all

    # Restaurar backup
    backup_file = Dir.glob('db/backups/peludog_backup_*.sql.gz').last
    system("rake db:restore[#{backup_file}] RAILS_ENV=test")

    # Verificar datos restaurados
    assert_equal 1, User.count
    assert_equal 'Test', User.first.name
  end
end
```